Lecture L10

*Blind review — please score this lecture on row **L10** in the Excel form (1–10 per dimension).*

Introduction to Normalization Forms

Imagine a database where a single change to a customer's address requires updating multiple tables, leading to inconsistencies and errors. This is where database normalization comes in – a crucial process that helps minimize data redundancy and dependency. By the end of this lecture, we will be able to define and explain the purpose of database normalization, identify and apply the rules for various normal forms, analyze a given database schema, design a normalized database schema, and evaluate the trade-offs between data integrity, performance, and complexity in database design.

Database normalization is the process of organizing data in a database to minimize data redundancy and dependency. Normalization involves decomposing large tables into smaller tables while linking them through relationships, which helps to improve data integrity and reduce data anomalies. To illustrate this, consider a library's catalog system, where books are organized by author, title, and genre. If we were to store all this information in a single table, it would be akin to having a single, massive bookshelf with no clear organization – making it difficult to find specific books or update information.

Having defined the concept of database normalization, we now consider a simple example to understand the need for normalization. Suppose we have a table that stores information about students, including their names, ages, and courses taken. If each row represents a single student, and each student has multiple courses, then the table is not normalized because the courses column contains multiple values. Try to design a simple database schema for this example and identify the potential issues that arise from not normalizing the data.

First Normal Form (1NF)

A relation is said to be in 1NF if and only if each cell in the relation contains a single, atomic value. To illustrate this, consider a table where each row represents a single student, and each student has multiple courses. The table is not in 1NF because the courses column contains multiple values. To convert this table to 1NF, we need to create a separate row for each course taken by each student. Think of it like a receipt, where each item purchased is listed on a separate line – making it easier to calculate the total cost and manage inventory.

For instance, suppose we have a table with the following data: | Student Name | Age | Courses | | — | — | — | | John | 20 | Math, Science, History | To convert this table to 1NF, we would create separate rows for each course: | Student Name | Age | Course | | — | — | — | | John | 20 | Math | | John | 20 | Science | | John | 20 | History | Now, try to apply this concept to a real-world scenario, such as a customer ordering multiple products from an e-commerce website. How would you design the database schema to ensure that each order is properly normalized?

Second Normal Form (2NF)

A relation is said to be in 2NF if and only if it is in 1NF and each non-key attribute in the relation depends on the entire primary key. To understand this, consider an example where we have a table that stores information about orders, including the order date, customer name, and order total. If the primary key is the combination of order date and customer name, then the order total should depend on both the order date and the customer name, not just one of them. Think of it like a contract, where the terms and conditions depend on both the signing date and the parties involved.

Suppose we have a table with the following data: | Order Date | Customer Name | Order Total | | — | — | — | | 2022-01-01 | John | 100 | | 2022-01-01 | Jane | 200 | | 2022-01-02 | John | 50 | To determine if this table is in 2NF, we need to check if the order total depends on both the order date and the customer name. If it does, then the table is in 2NF. Now, consider a scenario where the order total depends only on the customer name – how would you redesign the table to meet the 2NF criteria?

Third Normal Form (3NF)

A relation is said to be in 3NF if and only if it is in 2NF and there are no transitive dependencies. To understand this, consider an example where we have a table that stores information about employees, including their names, ages, and departments. If the department name depends on the employee's name, then the department name should be moved to a separate table. Think of it like a company's organizational chart, where each department has its own manager and team – making it easier to manage and update the hierarchy.

Suppose we have a table with the following data: | Employee Name | Age | Department | | — | — | — | | John | 30 | Sales | | Jane | 25 | Marketing | | Bob | 40 | Sales | To determine if this table is in 3NF, we need to check if there are any transitive dependencies. If the department name depends on the employee's name, then we need to move the department name to a separate table. Now, try to apply this concept to a real-world scenario, such as a university's course catalog – how would you design the database schema to ensure that each course is properly normalized?

Boyce-Codd Normal Form (BCNF)

A relation is said to be in BCNF if and only if it is in 3NF and there are no transitive dependencies, and a composite key is not required. To understand this, consider an example where we have a table that stores information about orders, including the order date, customer name, and order total. If the primary key is the combination of order date and customer name, and the order total depends on both the order date and the customer name, then the table is in BCNF. Think of it like a unique identifier, where each order has a distinct combination of date and customer – making it easier to track and manage orders.

Suppose we have a table with the following data: | Order Date | Customer Name | Order Total | | — | — | — | | 2022-01-01 | John | 100 | | 2022-01-01 | Jane | 200 | | 2022-01-02 | John | 50 | To determine if this table is in BCNF, we need to check if it is in 3NF and if there are no transitive dependencies, and if a composite key is not required. Now, consider a scenario where the order total depends on a single attribute, such as the customer name – how would you redesign the table to meet the BCNF criteria?

Fourth Normal Form (4NF)

A relation is said to be in 4NF if and only if it is in BCNF and there are no multi-level dependencies. To understand this, consider an example where we have a table that stores information about employees, including their names, ages, and departments. If an employee can work on multiple projects, and each project has multiple employees, then the table is not in 4NF because there is a multi-level dependency. Think of it like a complex network, where each employee is connected to multiple projects, and each project is connected to multiple employees – making it difficult to manage and update the relationships.

Suppose we have a table with the following data: | Employee Name | Age | Department | Project | | — | — | — | — | | John | 30 | Sales | Project A | | John | 30 | Sales | Project B | | Jane | 25 | Marketing | Project A | To determine if this table is in 4NF, we need to check if it is in BCNF and if there are no multi-level dependencies. If there is a multi-level dependency, then we need to move the project information to a separate table. Now, try to apply this concept to a real-world scenario, such as a social

media platform – how would you design the database schema to ensure that each user's relationships are properly normalized?

Denormalization

Denormalization is the process of intentionally violating the rules of normalization in order to improve performance. Denormalization can be used to reduce the number of joins required to retrieve data, which can improve query performance. However, denormalization can also lead to data redundancy and inconsistencies, which can make it more difficult to maintain data integrity. Think of it like a shortcut, where we sacrifice some data integrity for the sake of speed – but we need to be careful not to get lost in the process.

Consider an example where we have a table that stores information about customers, including their names, addresses, and order history. If we need to retrieve the customer's order history frequently, it may be more efficient to denormalize the table and store the order history in a separate column, rather than joining the customer table with the order table. Now, try to weigh the pros and cons of denormalization in this scenario – is the improved performance worth the potential risks to data integrity?

Practical Use of Normal Forms

In practice, database designers often aim to achieve BCNF or 3NF for every relation schema. However, higher normal forms, such as 4NF and 5NF, are not always necessary, and may even be questionable in some cases. The choice of normal form depends on the specific requirements of the database and the trade-offs between data integrity, performance, and complexity. Consider an example where we are designing a database for an e-commerce application. We need to store information about customers, orders, and products. To achieve BCNF, we would create separate tables for customers, orders, and products, and link them through relationships. This design would improve data integrity and reduce data anomalies, but may also increase the complexity of the database.

Having considered the practical use of normal forms, we now understand the importance of choosing the appropriate normal form for a given database design. The choice of normal form depends on the specific requirements of the database and the trade-offs between data integrity, performance, and complexity. By applying the rules of normalization and considering the practical use of normal forms, we can design a database that meets the needs of the application and ensures data integrity and consistency. Now, try to design a database schema for a real-world application, such as a university's student information system – what normal form would you aim to achieve, and why?